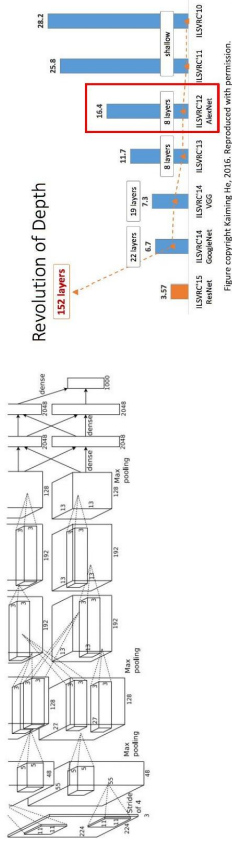
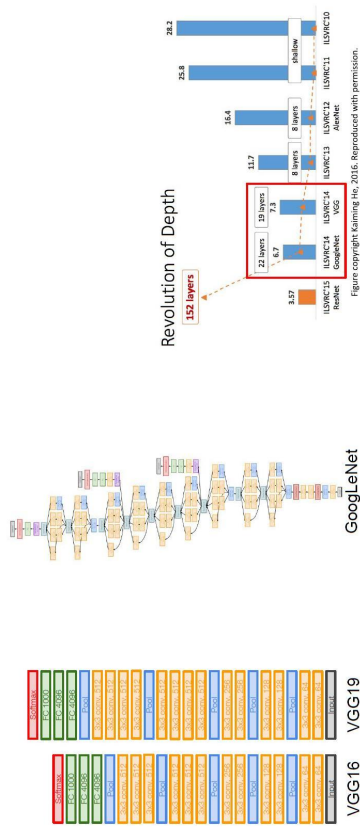


Last Time: CNN Architectures

AlexNet



Last Time: CNN Architectures



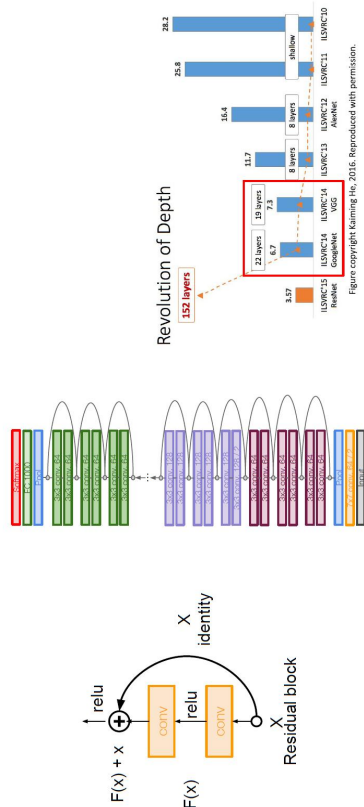
Announcements

- Finish MQ

- Please kindly be reminded that you are encouraged to provide your feedback to the module INT305 through the Module Feedback Questionnaire.
- Please take your time to help improving the quality of this module by sharing your comments.

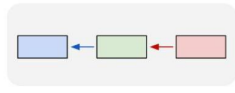
Today: Recurrent Neural Networks

Last Time: CNN Architectures



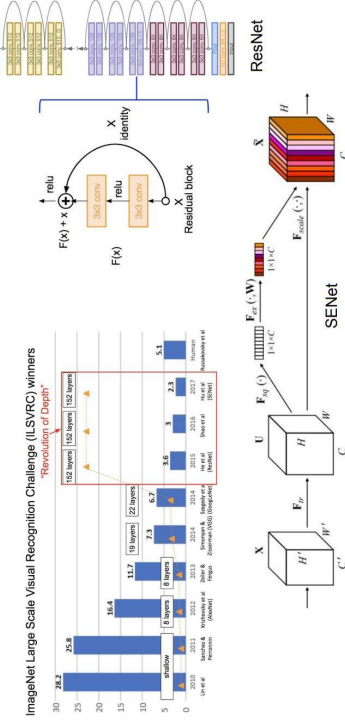
“Vanilla” Neural Network

one to one

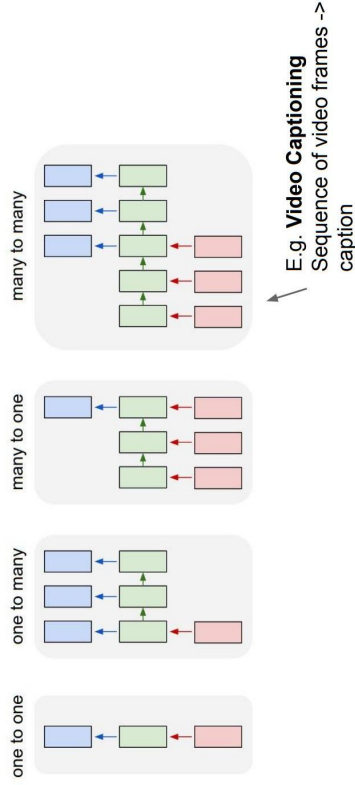


Vanilla Neural Networks

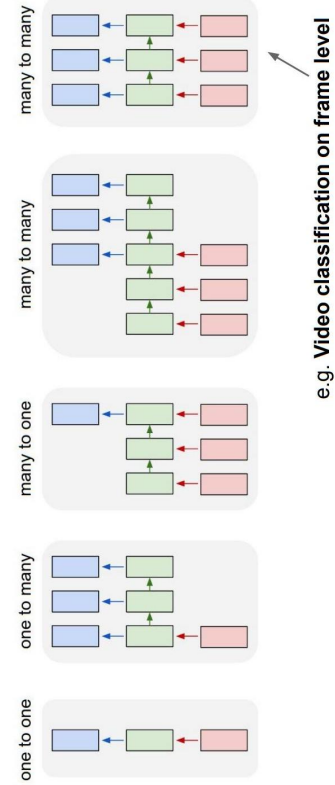
Last Time: CNN Architectures



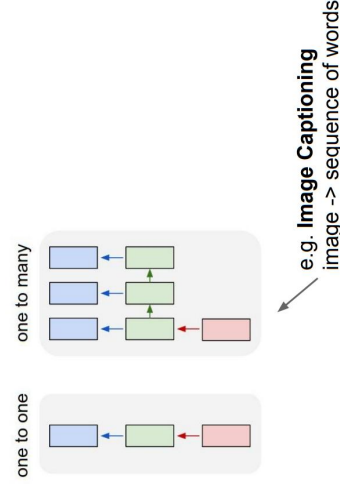
Recurrent Neural Networks: Process Sequence



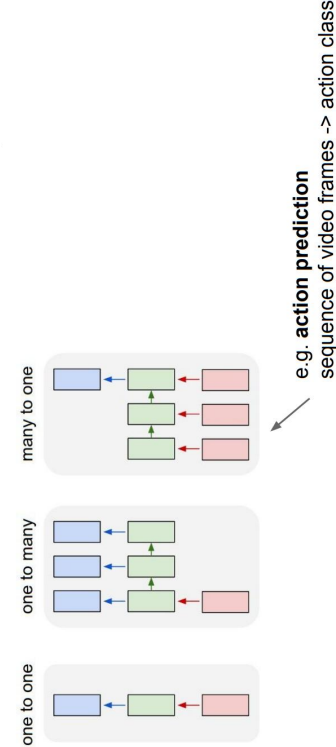
Recurrent Neural Networks: Process Sequence



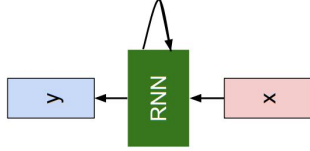
Recurrent Neural Networks: Process Sequence



Recurrent Neural Networks: Process Sequence



Recurrent Neural Network



Key idea: RNNs have an "internal state" that is updated as a sequence is processed



Why existing convnets are insufficient?

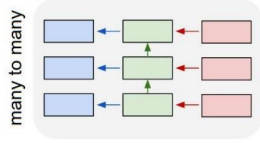
Variable sequence length inputs and outputs!



Krishna, Hata, Ren, Fei-Fei, Niebles. Dense captioning Events in Videos. ICCV 2019

Recurrent Neural Network

Let's start with a task that takes a variable input and produces an output at every step



RNN output generation

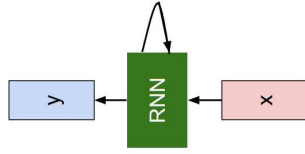
We can process a sequence of vectors $\mathbf{x}$ by applying a **recurrence formula** at every time step:

$$y_t = f_{W_{hy}}(h_t)$$

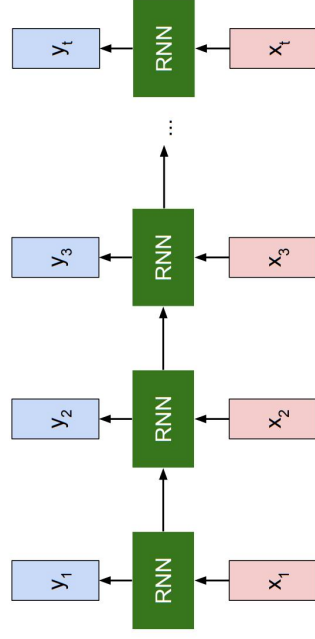
output

another function with parameters W_o

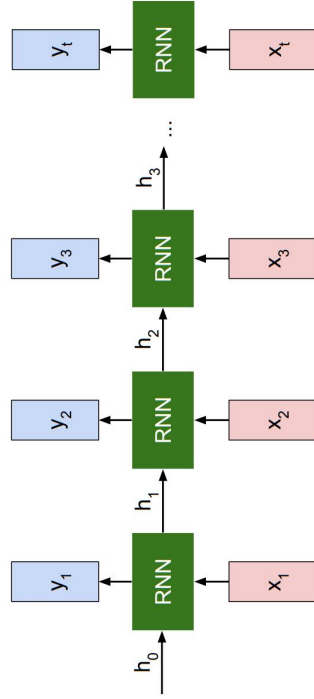
new state



Unrolled RNN



Recurrent Neural Network



RNN hidden state update

We can process a sequence of vectors $\mathbf{x}$ by applying a **recurrence formula** at every time step:

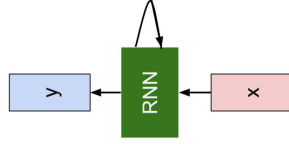
$$h_t = f_W(h_{t-1}, x_t)$$

new state

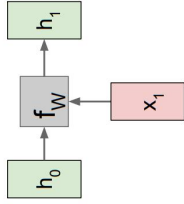
some function with parameters W

old state

input vector at some time step



RNN: Computational Graph

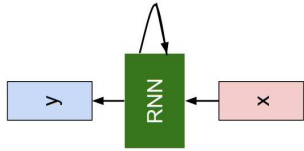


Recurrent Neural Network

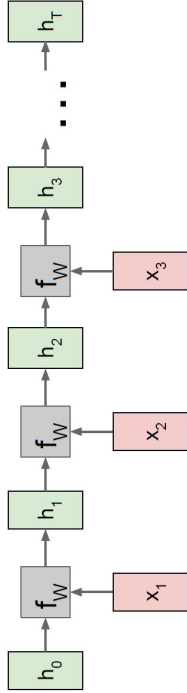
We can process a sequence of vectors $\mathbf{x}$ by applying a **recurrence formula** at every time step:

$$h_t = f_W(h_{t-1}, x_t)$$

Notice: the same function and the same set of parameters are used at every time step.

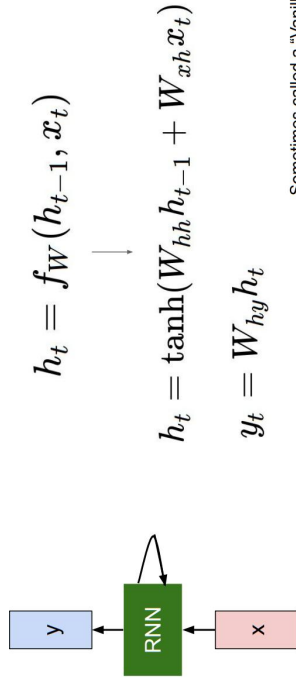


RNN: Computational Graph



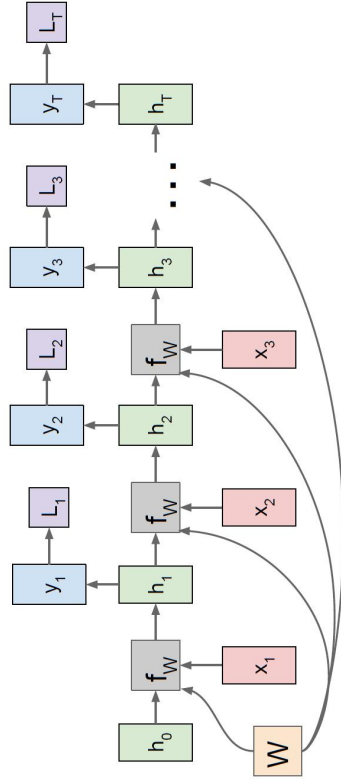
(Simple) Recurrent Neural Network

The state consists of a single "hidden" vector $\mathbf{h}$:

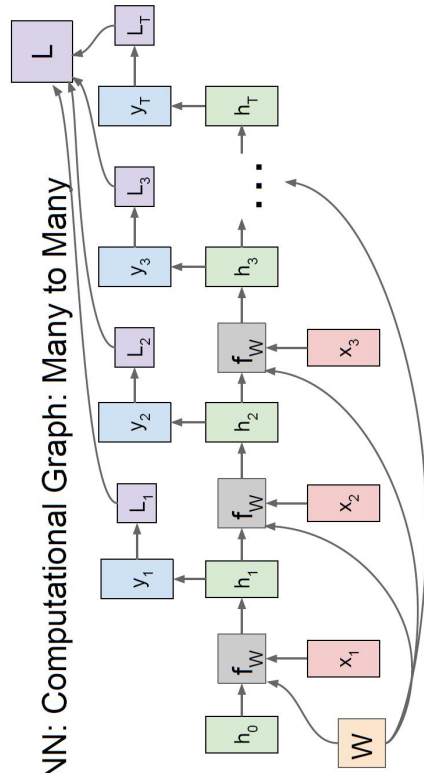


Sometimes called a "Vanilla RNN" or an "Elman RNN" after Prof. Jeffrey Elman

RNN: Computational Graph: Many to Many

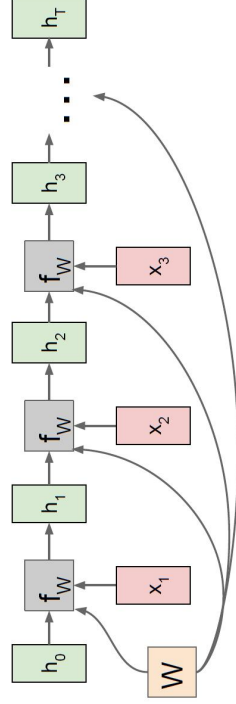


RNN: Computational Graph: Many to Many

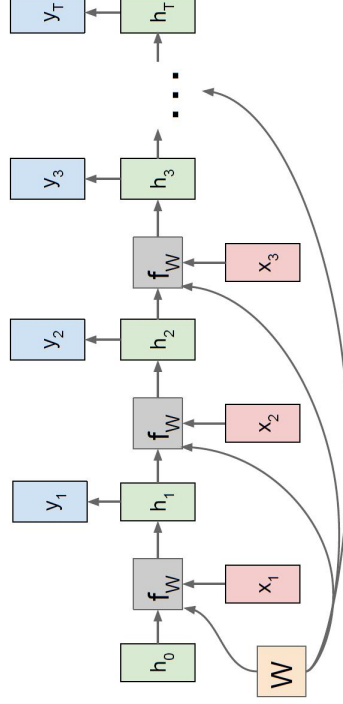


RNN: Computational Graph

Re-use the same weight matrix at every time-step

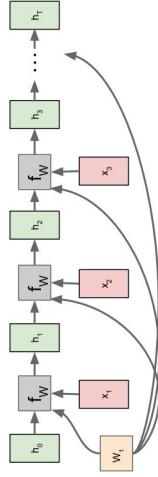


RNN: Computational Graph: Many to Many



Sequence to Sequence: Many-to-one + one-to-many

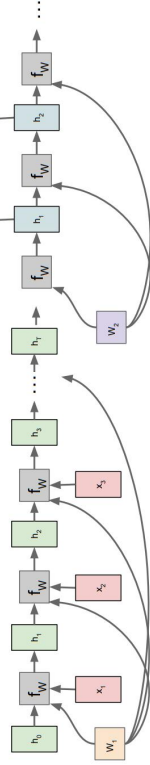
Many to one: Encode input sequence in a single vector



Sutskever et al. "Sequence to Sequence Learning with Neural Networks", NIPS 2014

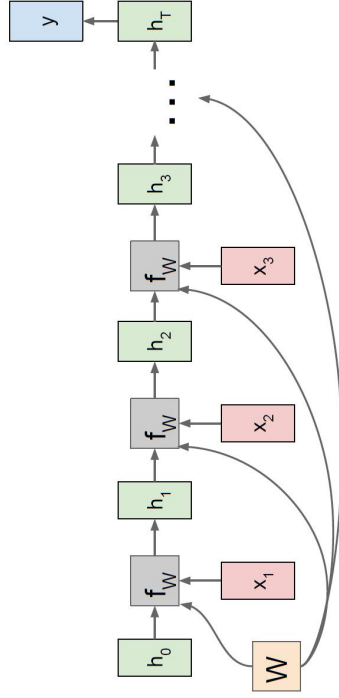
Sequence to Sequence: Many-to-one + one-to-many

Many to one: Encode input sequence in a single vector

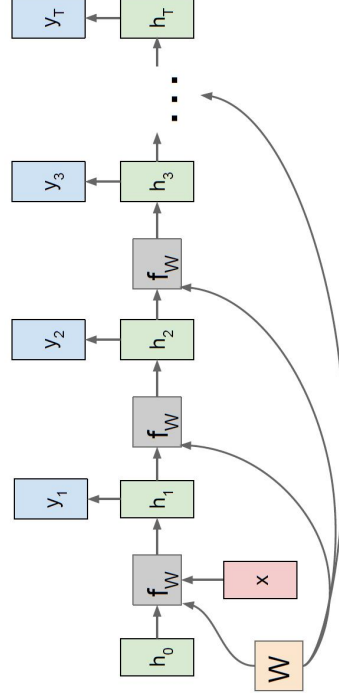


Sutskever et al. "Sequence to Sequence Learning with Neural Networks", NIPS 2014

RNN: Computational Graph: Many to One



RNN: Computational Graph: One to Many

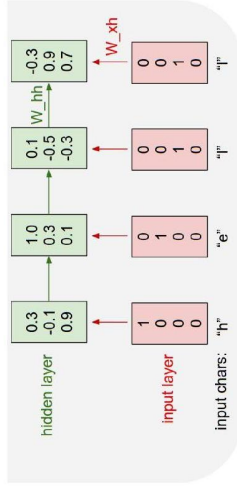


Example:
Character-level
Language Model

Vocabulary:
[h,e,l,o]

Example training
sequence:
"hello"

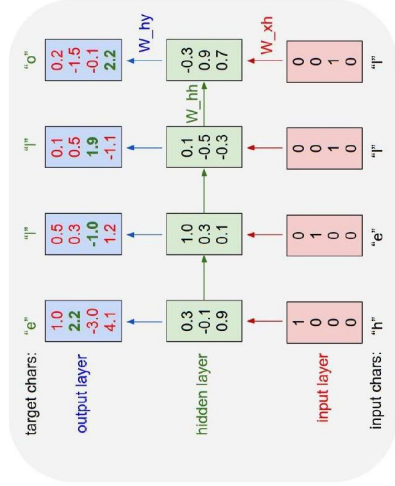
$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$



Example:
Character-level
Language Model

Vocabulary:
[h,e,l,o]

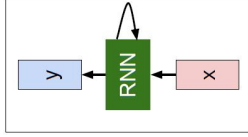
Example training
sequence:
"hello"



Example:
Character-level
Language Model

Vocabulary:
[h,e,l,o]

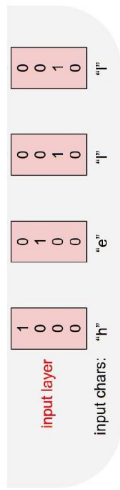
Example training
sequence:
"hello"



Example:
Character-level
Language Model

Vocabulary:
[h,e,l,o]

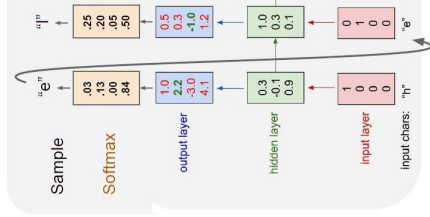
Example training
sequence:
"hello"



Example: Character-level Language Model Sampling

Vocabulary:
[h,e,l,o]

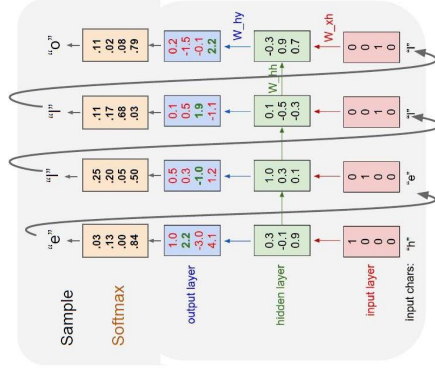
At test-time sample
characters one at a time,
feed back to model



Example: Character-level Language Model Sampling

Vocabulary:
[h,e,l,o]

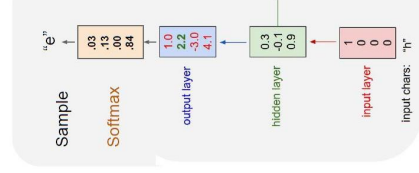
At test-time sample
characters one at a time,
feed back to model



Example: Character-level Language Model Sampling

Vocabulary:
[h,e,l,o]

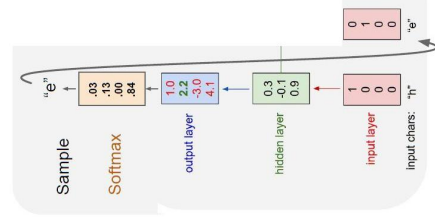
Example training
sequence:
"hello"



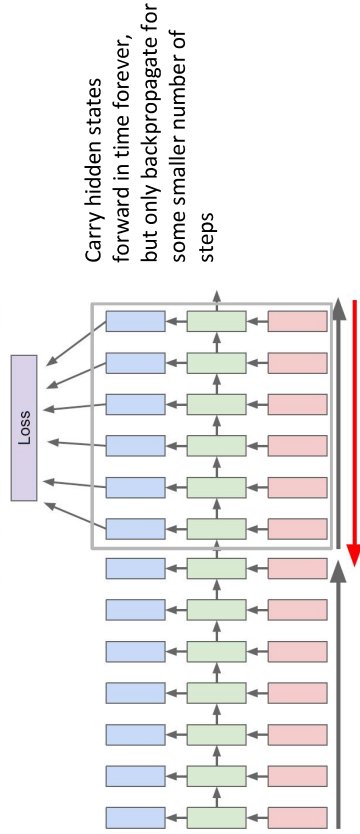
Example: Character-level Language Model Sampling

Vocabulary:
[h,e,l,o]

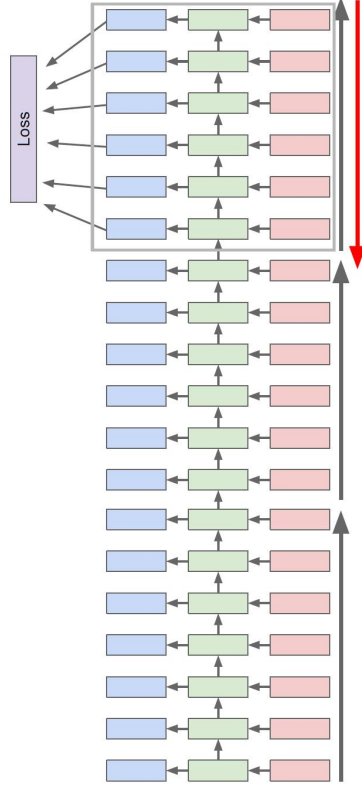
At test-time sample
characters one at a time,
feed back to model



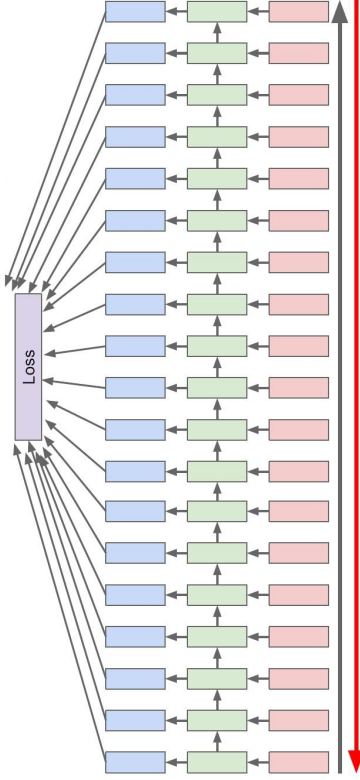
Truncated Backpropagation through time



Truncated Backpropagation through time

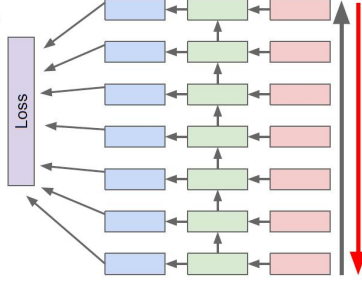


Backpropagation through time



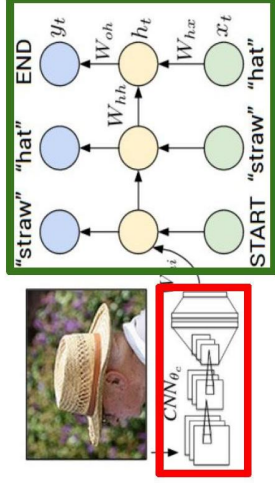
Forward through entire sequence to compute loss, then backward through entire sequence to compute gradient

Truncated Backpropagation through time



Run forward and backward through chunks of the sequence instead of whole sequence

Recurrent Neural Network

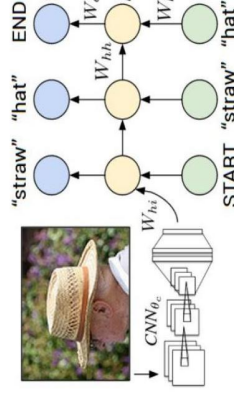


Convolutional Neural Network

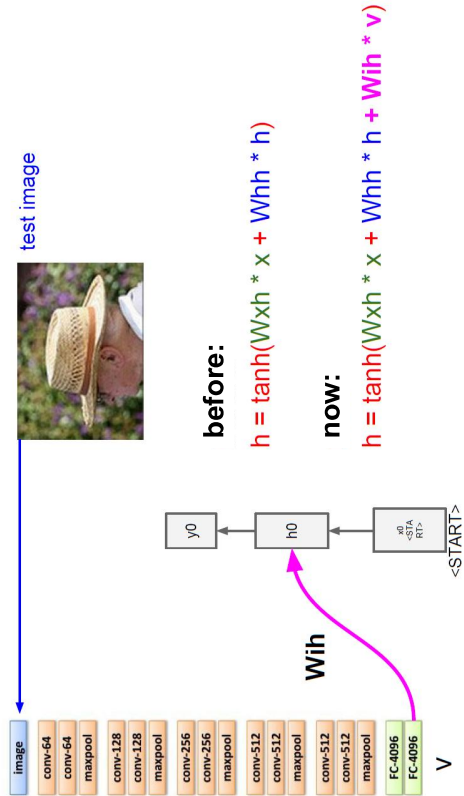


test image

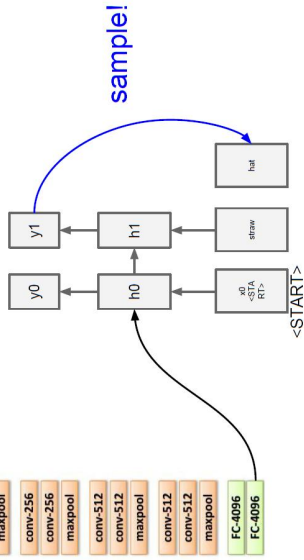
Image Captioning



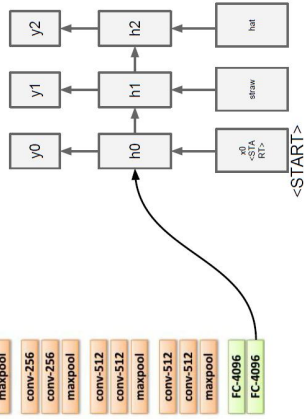
Explain Images with Multimodal Recurrent Neural Networks, Mao et al.
Deep Visual-Semantic Alignments for Generating Image Descriptions, Karpathy and Fei-Fei
Show and Tell: A Neural Image Caption Generator, Vinyals et al.
Long-term Recurrent Convolutional Networks for Visual Recognition and Description, Donahue et al.
Learning a Recurrent Visual Representation for Image Caption Generation, Chen and Zitnick



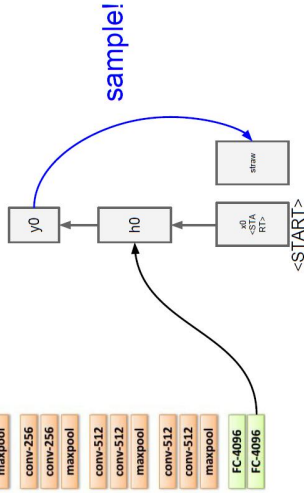
test image



test image



test image



test image

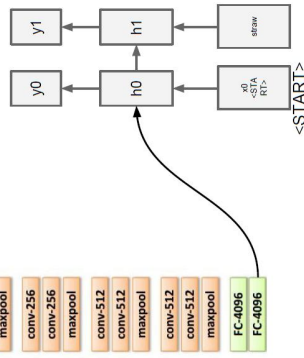
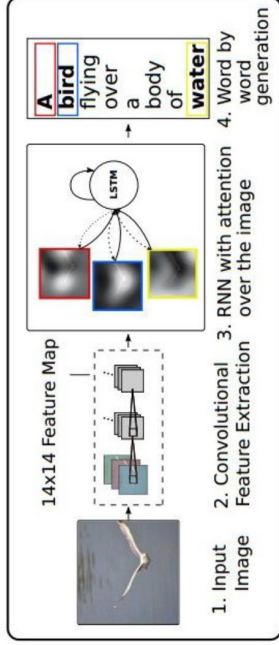


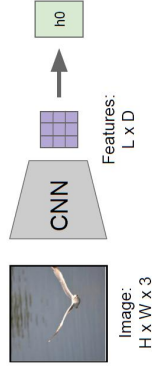
Image Captioning with Attention

RNN focuses its attention at a different spatial location when generating each word



Xu et al. "Show, Attend, and Tell: Neural Image Caption Generation with Visual Attention", ICML, 2015
Figure copyright Kevin Xu, Jimmy Lei Ba, Jaime Kiros, Kyunghyun Cho, Adam Cheurwater, Ruslan Samulin, Richard S. Zemel, and Yoshua Bengio, 2015. Reproduced with permission.

Image Captioning with Attention



Xu et al. "Show, Attend, and Tell: Neural Image Caption Generation with Visual Attention", ICML, 2015

Image Captioning: Example Results

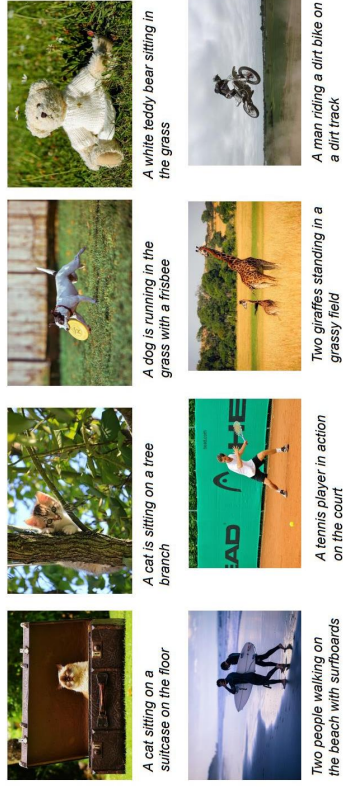


Image Captioning: Failure Cases

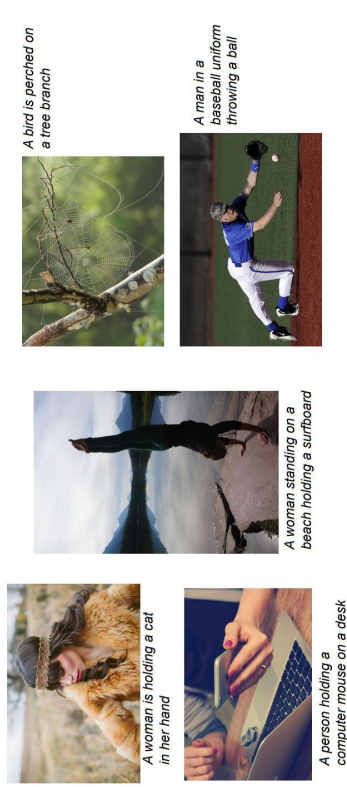
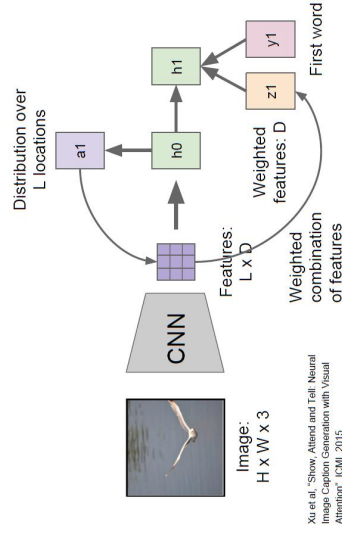
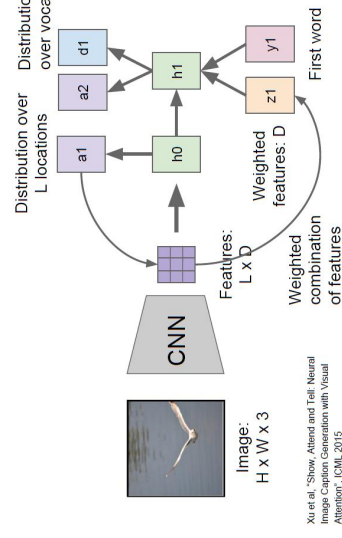


Image Captioning with Attention



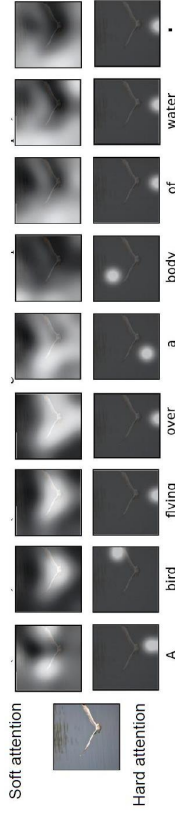
Xu et al., "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention" ICML 2015

Image Captioning with Attention



Xu et al, "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention". ICMIL 2015

Image Captioning with Attention



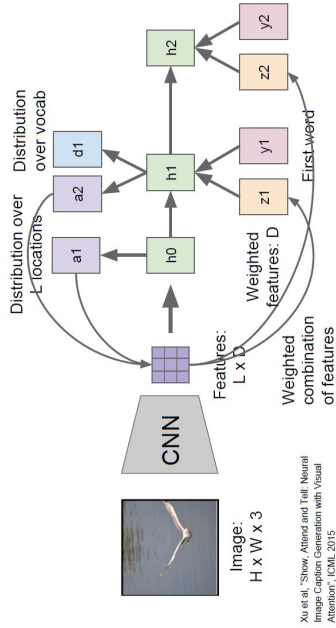
Xu et al. "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", ICML, 2015.
Figure copyright © Xu, Jimmy and Ba, James Kim, Aravindh Mahendran, Aaron Courville, Naveen Vasudevan, Bharat Ramakrishna, Richard S. Zemel, and Yoshua Bengio, 2015. Reproduced with permission.

Image Captioning with Attention



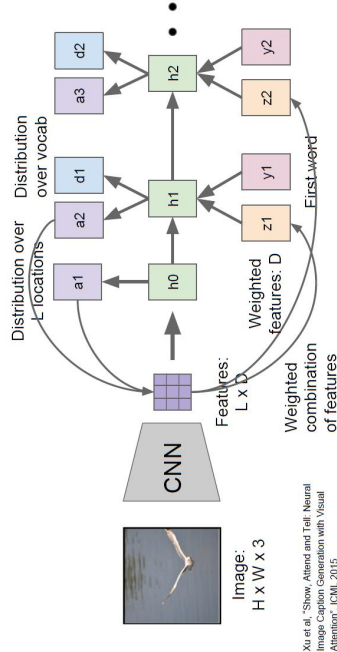
Xu et al. "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", ICML, 2015.
Figure copyright © Xu, Jimmy and Ba, James Kim, Aravindh Mahendran, Aaron Courville, Naveen Vasudevan, Bharat Ramakrishna, Richard S. Zemel, and Yoshua Bengio, 2015. Reproduced with permission.

Image Captioning with Attention



Xu et al. "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", ICML, 2015.

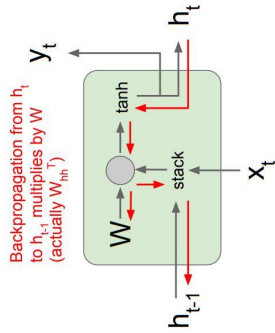
Image Captioning with Attention



Xu et al. "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", ICML, 2015.

Vanilla RNN Gradient Flow

Bengio et al. "Learning long-term dependencies with gradient descent is difficult". IEEE Transactions on Neural Networks, 1994
Pascanu et al. "On the difficulty of training recurrent neural networks". ICML, 2013

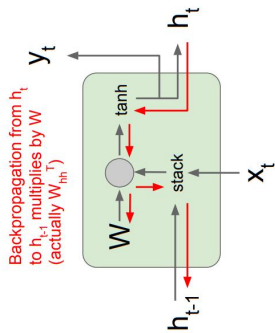


$$\begin{aligned} h_t &= \tanh(W_{hh}h_{t-1} + W_{xh}x_t) \\ &= \tanh\left((W_{hh} \quad W_{hx}) \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \\ &= \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \end{aligned}$$

Hochreiter and Schmidhuber, "Long Short Term Memory", Neural Computation 1997

Vanilla RNN Gradient Flow

Bengio et al. "Learning long-term dependencies with gradient descent is difficult". IEEE Transactions on Neural Networks, 1994
Pascanu et al. "On the difficulty of training recurrent neural networks". ICML, 2013



$$\begin{aligned} h_t &= \tanh(W_{hh}h_{t-1} + W_{xh}x_t) \\ &= \tanh\left((W_{hh} \quad W_{hx}) \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \\ &= \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \end{aligned}$$

$$\frac{\partial h_t}{\partial h_{t-1}} = \tanh'(W_{hh}h_{t-1} + W_{xh}x_t)W_{hh}$$

Long Short Term Memory (LSTM)

Vanilla RNN

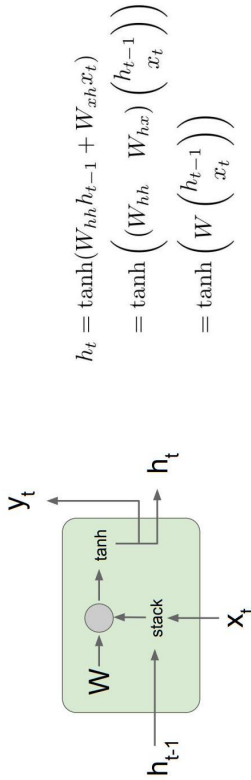
$$h_t = \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)$$

LSTM

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$
$$\begin{aligned} c_t &= f \odot c_{t-1} + i \odot g \\ h_t &= o \odot \tanh(c_t) \end{aligned}$$

Bengio et al. "Learning long-term dependencies with gradient descent is difficult". IEEE Transactions on Neural Networks, 1994
Pascanu et al. "On the difficulty of training recurrent neural networks". ICML, 2013

Vanilla RNN Gradient Flow

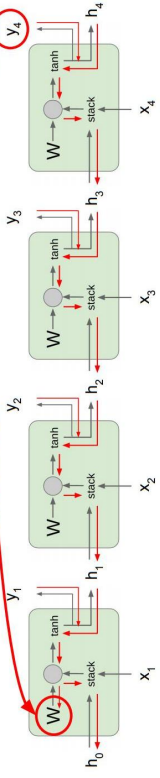


$$\begin{aligned} h_t &= \tanh(W_{hh}h_{t-1} + W_{xh}x_t) \\ &= \tanh\left((W_{hh} \quad W_{hx}) \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \\ &= \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \end{aligned}$$

Vanilla RNN Gradient Flow

Gradients over multiple time steps:

Bengio et al. "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al. "On the difficulty of training recurrent neural networks", ICML 2013

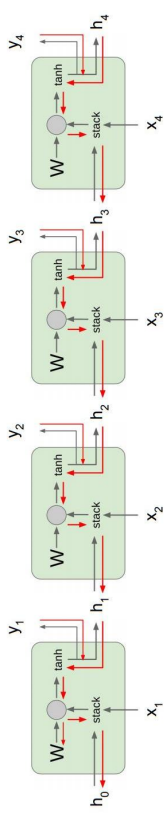


$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \frac{\partial h_t}{\partial h_{t-1}} \cdots \frac{\partial h_1}{\partial W}$$

Vanilla RNN Gradient Flow

Bengio et al. "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al. "On the difficulty of training recurrent neural networks", ICML 2013

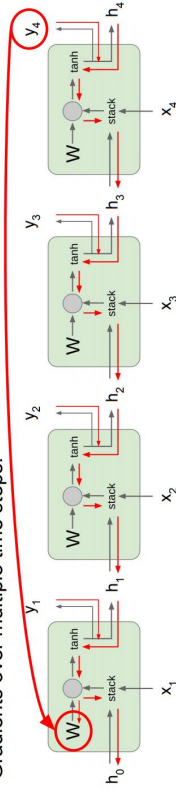


$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

Vanilla RNN Gradient Flow

Gradients over multiple time steps:

Bengio et al. "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al. "On the difficulty of training recurrent neural networks", ICML 2013

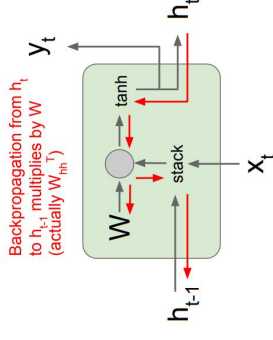


$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \frac{\partial h_t}{\partial h_{t-1}} \cdots \frac{\partial h_1}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^T \frac{\partial h_t}{\partial h_{t-1}} \right) \frac{\partial h_1}{\partial W}$$

Vanilla RNN Gradient Flow

Bengio et al. "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al. "On the difficulty of training recurrent neural networks", ICML 2013



$$\begin{aligned} h_t &= \tanh(W_{hh}h_{t-1} + W_{hx}x_t) \\ &= \tanh\left((W_{hh} \quad W_{hx}) \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \\ &= \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \end{aligned}$$

$$\frac{\partial h_t}{\partial h_{t-1}} = \tanh'(W_{hh}h_{t-1} + W_{hx}x_t)W_{hh}$$

Long Short Term Memory (LSTM)

Vanilla RNN

$$h_t = \tanh \left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} \right)$$

LSTM

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

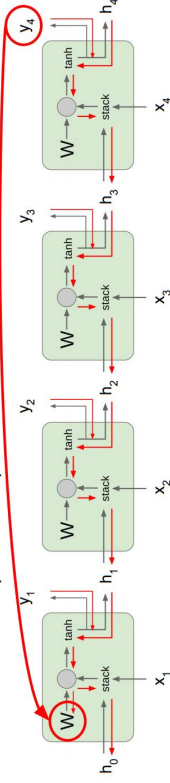
$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Hochreiter and Schmidhuber, "Long Short-Term Memory", Neural Computation

Vanilla RNN Gradient Flow

Gradients over multiple time steps:

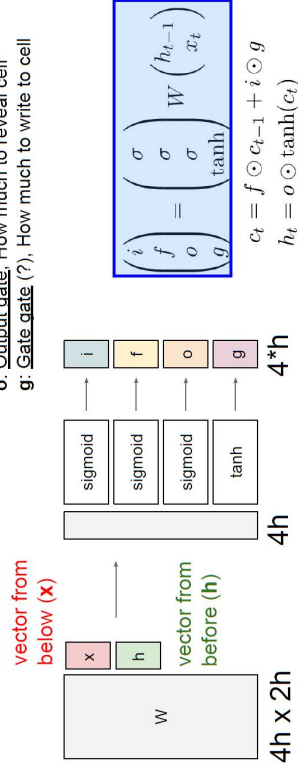


$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W} \quad \frac{\partial h_t}{\partial h_{t-1}} = \tanh'(W_{hh} h_{t-1} + W_{hx} x_t) W_{hh}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \frac{\partial h_t}{\partial h_{t-1}} \dots \frac{\partial h_1}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^T \frac{\partial h_t}{\partial h_{t-1}} \right) \frac{\partial h_1}{\partial W}$$

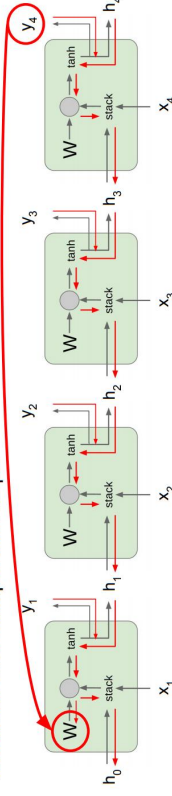
Long Short Term Memory (LSTM)

- i: Input gate, whether to write to cell
- f: Forget gate, Whether to erase cell
- o: Output gate, How much to reveal cell
- g: Gate gate (?), How much to write to cell



Vanilla RNN Gradient Flow

Gradients over multiple time steps:



$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W} \quad \text{Almost always } < 1 \quad \text{Vanishing gradients}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^T \frac{\partial h_t}{\partial h_{t-1}} \right) \frac{\partial h_1}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^T \tanh'(W_{hh} h_{t-1} + W_{hx} x_t) \right) W_{hh} \frac{\partial h_1}{\partial W}$$

Bengio et al. "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al. "On the difficulty of training recurrent neural networks", ICML 2013

LSTM

Long Short Term Memory (LSTM)

[Hochreiter et al., 1997]

